



← Back to student guides

ClearML

Experiment tracking

3 levels

116 sections



The Complete Guide

Taught at three levels, written three ways. Pick Beginner, Mid-level, or Senior, then read the full explanation, a fast interview review, or the practical tips and traps for that level.

TASK

QUEUE

AGENT

MODEL

16

sections

34

examples





metrics and artifacts on purpose, compare twenty runs in a table, version a dataset, clone a run and re-launch it on a GPU box you never SSH into, and answer "what exactly produced this model?" with a task id. Mid-level and Senior take the same topics further; nothing here is thrown away.

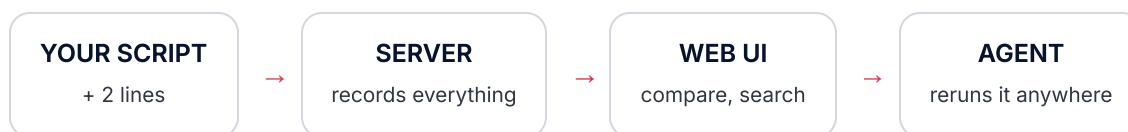
Each section ends with a **Try it** task. Do them as you go. They take a few minutes each, and these ideas only stick once you have watched a scalar appear in the web UI while your script is still running.

What ClearML is, and the problem it solves

You trained a model on Tuesday. It scored 0.94. On Thursday you cannot reproduce it. The notebook has moved on, the learning rate in the cell is not the one you ran, the dataset folder has been "cleaned up", and the only record of 0.94 is a screenshot in Slack.

Discipline was never the problem. The tooling was: **nothing was watching**. Every fact about that run lived in your head, your terminal scrollback, and a filename.

ClearML watches. You add two lines to your script and it records the code, the git commit, the uncommitted diff, the installed packages, the command-line arguments, every metric you print to TensorBoard, the model file you saved, and the machine it ran on, all into a server you and your team can query.



ClearML's central trick is that a recorded run is executable. Other tools log an experiment so you can look at it. ClearML logs enough (repository, commit, diff, packages, arguments, environment) that a worker can rebuild the environment and run it again on a different machine. That single property is what turns a log into a workflow.

That gives you four things, and they are the reason ClearML exists rather than a spreadsheet and a naming convention:



Automatic experiment tracking

Two lines. Code, config, packages, metrics, and models get recorded without you writing logging calls.

Runs you can re-execute

Clone any past run, change one parameter, and send it to a queue. No SSH, no environment setup.



Versioned datasets

Immutable, content-addressed dataset versions with parents, so "which data?" has an id for an answer.



Pipelines and orchestration

Chain those runs into a graph, run each step on a different queue, and cache the steps that did not change.

The vocabulary is small, and getting it straight now saves confusion later:

Term	What it is
Task	One recorded run: an experiment, a dataset version, a pipeline step. The core object
Project	A folder of tasks. Supports <code>/</code> for nesting, like <code>research/nlp/ner</code>
Server	The API server, web server, and file server that store and show everything
Agent	A worker process that pulls tasks off a queue and executes them
Queue	A named, ordered list of tasks waiting for an agent
Artifact	Any file or object you attach to a task: a dataframe, a plot, a checkpoint
Model	A first-class artifact with its own registry entry, tags, and lineage
Dataset	A special task that holds an immutable, versioned collection of files

You need little to follow along: Python, a script that trains something small, and a free account on the hosted server. A tiny model on a tiny dataset is the best place to start, because a broken run costs you thirty seconds.



1. Find your most recent training script. Write down, from memory, the exact learning rate, batch size, and dataset it last ran with.
2. Now check the script and see whether you were right.
3. Look for the model file it produced and try to name the git commit that trained it.
4. Count how many folders on your disk match `*_v2*` , `*_final*` , or `runs/` .

Expected result – at least one of those three questions is unanswerable, and usually all three. That is the gap ClearML fills, and the third one, tying a model file to a commit, is the one that hurts most in a code review six months later.

The architecture: four parts, three data stores

ClearML has more moving pieces than a single library, and knowing which piece does what makes every error message readable.

YOUR TRAINING PROCESS

SDK: `Task.init()`

Patches frameworks at import time, records code, diff, packages, args; streams metrics



SERVER: API :8008 • WEB :8080 • FILES :8081

MongoDB

Task metadata: params, status, queues. "Task list is slow" lives here

Elasticsearch

Metrics and console logs. "Scalars are slow", and it fills up first

Redis



EXECUTION: OPTIONAL, AND THE HALF PEOPLE FIND CONFUSING

Queue

A named, ordered list. A task sits here until an agent claims it

Agent

Rebuilds the env, clones the commit, applies the diff, runs the entry point

Artifact storage

File server, or S3 via `output_uri` : use S3 for anything large

Learn the top two lanes first. Tracking needs no agent anywhere: install the SDK, run your script, get a full record. The bottom lane is the second half of the story, `re-executing` a recorded run elsewhere, and trying to learn both at once is why people find ClearML complicated.

Part	What it does	Where it runs
SDK (<code>pip install clearml</code>)	Patches your frameworks, sends metadata and metrics, uploads files	Inside your training process
API server	Stores tasks, parameters, metrics, and queues. Everything talks to it	Port 8008
Web server	The UI you compare experiments in. Talks only to the API server	Port 8080
File server	Stores artifacts, models, and debug images when you have no S3 bucket	Port 8081
Agent (<code>pip install clearml-agent</code>)	Pulls a task from a queue, rebuilds its environment, runs it	Any machine with the code's dependencies

Behind the API server sit three data stores, and knowing which is which explains later performance advice: **MongoDB** holds task metadata, **Elasticsearch** holds metrics and console



You have two options for the server, and for learning there is only one sensible answer:

✓ **Hosted (app.clear.ml)**

- Free tier, no setup at all
- Signup, copy credentials, done in two minutes
- Right choice for learning and for small teams

✗ **Self-hosted**

- Docker Compose or Kubernetes, plus Mongo, Elasticsearch, Redis
- You own backups, upgrades, storage, and access control
- Necessary when data cannot leave your network; Senior covers it

The agent is where the confusion usually is

Nothing about tracking requires an agent. You can install the SDK, run your script, and get full tracking with no worker anywhere. The agent is for the second half of the story: **re-executing** a recorded run somewhere else. Learn tracking first, then add the agent; trying to learn both at once is why people find ClearML complicated.

► **TRY IT**

1. Sign up at the hosted server and open the Workers & Queues page. It is empty. Nothing needs an agent yet.
2. Open Settings → Workspace and find the "Create new credentials" button. Do not click it yet.
3. Sketch the four parts above from memory and label which one your training script talks to.

Expected result – your script talks to the API server for metadata and the file server for files, and nothing else. The web server is for you, not for your code, and the agent is optional. Getting that picture straight now makes every later error message obvious.

Install and connect: credentials and config



BASH

Copy

```
pip install clearml
clearml-init
```

`clearml-init` asks you to paste a credentials block, which you copy from the web UI under **Settings** → **Workspace** → **Create new credentials**. It looks like this, and it is the only setup step in the whole guide:

pasted into clearml-init

Copy

```
api {
  web_server: https://app.clear.ml
  api_server: https://api.clear.ml
  files_server: https://files.clear.ml
  credentials {
    "access_key" = "ABC123..."
    "secret_key" = "xyz789..."
  }
}
```

That gets written to `~/clearml.conf`, a large, heavily commented file worth skimming once. The four lines that matter today:

Setting	Means
<code>api.api_server</code>	Where metadata goes
<code>api.files_server</code>	Where artifacts and models go by default
<code>api.credentials</code>	Your access key and secret key pair
<code>sdk.development.default_output_uri</code>	Optional: send models and artifacts to S3 instead of the file server

Every one of those can be overridden by an environment variable, which is how you configure CI and containers where there is no config file to edit:



```
export CLEARML_API_HOST=https://api.clear.ml
export CLEARML_WEB_HOST=https://app.clear.ml
export CLEARML_FILES_HOST=https://files.clear.ml
export CLEARML_API_ACCESS_KEY=ABC123...
export CLEARML_API_SECRET_KEY=xyz789...
```

Your secret key is a full-access credential

A ClearML access/secret pair can read every experiment in your workspace and enqueue tasks that run code on your agents. Treat it exactly like a cloud access key: never commit `clearml.conf`, never paste it into a notebook you will share, and use environment variables in CI so there is no file to leak. Senior covers scoped credentials and service accounts properly.

Confirm it works before writing any code:

BASH

Copy

```
clearml-init          # re-run any time to reconfigure
python -c "from clearml import Task; print(Task.get_projects()[:3])"
```

► TRY IT

1. Run `pip install clearml` then `clearml-init` and paste your credentials.
2. Open `~/clearml.conf` and find `files_server`, then find `sdk.development` and skim the comments.
3. Run the one-line Python check above and confirm you get a list back rather than an authentication error.
4. Add `clearml.conf` to your global gitignore right now: `echo "clearml.conf" >> ~/.gitignore_global`.

Expected result – a working connection and a config file you have looked at. That last step takes five seconds and prevents the most common ClearML security incident, which is a credentials block committed to a public repository.



This is the whole beginner story in two lines. Start from a script with no ClearML in it:

train.py (before)

Copy

```
import numpy as np
from sklearn.datasets import load_iris
from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import train_test_split

n_estimators = 100
max_depth = 4

X, y = load_iris(return_X_y=True)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

model = RandomForestClassifier(n_estimators=n_estimators, max_depth=max_depth)
model.fit(X_train, y_train)
print("accuracy", model.score(X_test, y_test))
```

Here it is tracked:

train.py (after)

Copy

```
from clearml import Task

task = Task.init(project_name="iris-demo", task_name="random forest baseline")

import numpy as np
from sklearn.datasets import load_iris
from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import train_test_split

params = task.connect({"n_estimators": 100, "max_depth": 4})

X, y = load_iris(return_X_y=True)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

model = RandomForestClassifier(n_estimators=params["n_estimators"], max_depth=params["ma
model.fit(X_train, y_train)
```



Run it. The first thing it prints is the link that matters:

TEXT

Copy

```
ClearML Task: created new task id=8f2c4b19e0a7d3f1c6b8a2e4d7091f3b  
ClearML results page: https://app.clear.ml/projects/.../experiments/8f2c4b19.../output/1
```

Open that link while the script is still running and you are watching the console output stream into the UI. Four things happened without you asking:

01

A task was created

With an id, a project, a name, a status of `running`, and a start time.

02

Your environment was recorded

Git remote, branch, commit, the diff of your uncommitted changes, and the installed packages the script imported.

03

Your stdout and stderr were captured

Streamed to the server, so the console tab is a live log you can read from your phone.

04

Your parameters were registered

Because you passed that dict through `task.connect`, they are now editable in the UI, which is what makes the run re-launchable.



1. Take the script above, run it, and open the results link while it is still running.
2. In the UI, visit each tab in turn: Execution, Configuration, Console, Scalars, Artifacts.
3. Under Execution, find the git commit and the "uncommitted changes" section. Make an unrelated edit to the file, rerun, and look again.
4. Under Configuration → Hyperparameters, confirm `n_estimators` and `max_depth` are listed with their values.

Expected result – the uncommitted diff is the surprise. ClearML stores the patch of your dirty working tree so a run from an uncommitted state is still reproducible, which is enormously useful and, as Senior covers, a place secrets can leak if you are careless.

What gets captured automatically, and what does not

The word ClearML uses for this is "automagic", and knowing its exact boundaries stops you from either duplicating work or expecting magic that is not there.

Category	What lands in the task
Code	Git remote, branch, commit, uncommitted diff, entry-point script and working directory
Environment	Python version, and the packages your script imported with their exact versions
Arguments	Every <code>argparse</code> flag, plus <code>click</code> , <code>fire</code> , and <code>hydra</code> configs
Metrics	Anything written to TensorBoard, TensorBoardX, or <code>matplotlib</code>
Models	Every checkpoint saved by PyTorch, TensorFlow, Keras, scikit-learn via joblib, XGBoost, LightGBM



Console	stdout and stderr, streamed
Machine	Hostname, CPU, GPU model, memory, and utilisation over time

That means an existing TensorBoard-instrumented project needs **no logging changes at all**:

already-instrumented.py

Copy

```
from clearml import Task
task = Task.init(project_name="vision", task_name="resnet baseline")

from torch.utils.tensorboard import SummaryWriter
writer = SummaryWriter()

for epoch in range(epochs):
    loss = train_one_epoch()
    writer.add_scalar("loss/train", loss, epoch)      # appears in ClearML Scalars
    torch.save(model.state_dict(), "checkpoint.pt")  # appears in ClearML Models
```

What is **not** captured, and this list matters:

Not captured	Why	What to do
Values you <code>print()</code> as metrics	A print is text, not a number with an iteration	<code>report_scalar</code>
Data read from a path outside version control	ClearML cannot know what was in it	Use a ClearML Dataset
Conda/apt/system libraries	Only Python imports are detected	Use docker mode on the agent
Environment variables	Deliberately excluded, because they hold secrets	Pass real config through parameters
Random seeds you never set	There is nothing to record	Set and connect them explicitly

You can turn parts of the magic off when it gets in the way:



```
task = Task.init(  
    project_name="vision",  
    task_name="resnet baseline",  
    auto_connect_frameworks={"matplotlib": False, "pytorch": True},  
    auto_connect_arg_parser=False,      # do not hoover up argparse  
    output_uri="s3://my-bucket/clearml", # send models and artifacts to S3  
)
```

► TRY IT

1. Add `Task.init` to a project of yours that already uses TensorBoard, and change nothing else.
2. Run it and confirm the scalars appear in ClearML's Scalars tab with the same titles and series.
3. Add an `argparse` flag, pass a value on the command line, and find it under Configuration → Hyperparameters → Args.
4. Now move `Task.init` to the bottom of the imports, rerun, and compare what was captured.

Expected result – the same curves with zero logging code, and, in step four, missing or partial scalars. Run that last experiment once and the "init first" rule becomes muscle memory rather than a thing you read.

Logging on purpose: the Logger

Automatic capture gets you a long way. The Logger is for everything you want recorded deliberately, and its API is small enough to memorise.

PYTHON

Copy

```
logger = task.get_logger()  
  
# A number that changes over training → a curve  
logger.report_scalar(title="loss", series="train", value=0.412, iteration=epoch)  
logger.report_scalar(title="loss", series="val", value=0.503, iteration=epoch)  
  
# A number that happens once → a table row, comparable across tasks
```



```
logger.report_table(title="confusion", series="test", iteration=0, table_plot=df)

# An image
logger.report_image(title="samples", series="epoch-3", iteration=3, local_path="grid.png")

# Free text into the task log
logger.report_text("Trained on dataset version 3fa1b2c")

# Any matplotlib figure
logger.report_matplotlib_figure(title="roc", series="test", iteration=0, figure=fig)
```

The **title** / **series** distinction is the one people get wrong, and it decides what your Scalars tab looks like:



► TRY IT

1. Instrument a loop with `report_scalar` using one `title` and two `series`. Watch the two lines land on one chart.
2. Now split them into two titles and see the difference in the Scalars tab. Put them back.
3. Add two `report_single_value` calls and find them under the Scalars tab's single-value section.
4. Report a matplotlib figure and confirm it appears under Plots, interactive rather than as a flat image.

Expected result – a Scalars tab you can read. The title/series experiment in steps one and two takes two minutes and permanently fixes the most common ClearML plotting mistake.

Hyperparameters: the editable surface of a task

Parameters are not just a record. In ClearML they are the **editable surface of a task**, the thing you change when you clone a run and re-launch it. That is why how you register them matters



PYTHON

Copy

```
# 1. A dict of scalars – the common case
params = task.connect({"lr": 3e-4, "batch_size": 64, "epochs": 20})
print(params["lr"])                # read from the returned dict, always

# 2. argparse – captured automatically, no call needed
parser = argparse.ArgumentParser()
parser.add_argument("--lr", type=float, default=3e-4)
args = parser.parse_args()          # already registered under "Args"

# 3. A nested config object or file – too big for a parameter list
config = task.connect_configuration(
    configuration="configs/model.yaml",
    name="model config",
)
```

The critical detail is that `connect` is **bidirectional**. When your code runs normally, `connect` uploads the dict to the server. When an agent runs a `clone` of that task, `connect` reads the values back from the server and overwrites your defaults. Same line of code, opposite direction.

Always read from the returned object, never from your literal

`params = task.connect({"lr": 3e-4})` then `use(params["lr"])` is correct. Calling `task.connect(cfg)` and then reading your original `cfg` variable works by luck for a



why it matters

Copy

```
# Locally: lr is 3e-4, uploaded to the server.
# In the UI you clone the task, edit lr to 1e-3, and enqueue.
# On the agent: connect() returns {"lr": 1e-3} – your 3e-4 default is ignored.
params = task.connect({"lr": 3e-4})
train(lr=params["lr"])          # runs with 1e-3, no code change
```

► TRY IT

1. Connect a dict of three hyperparameters and run the script.
2. In the UI, open Configuration → Hyperparameters and confirm all three are there and editable.
3. Add a YAML file with `connect_configuration` and find it under Configuration → Configuration Objects.
4. Deliberately read from your original literal instead of the returned dict, and note that nothing breaks

Expected result – yet

. The failure only appears once an agent runs a clone.

Expected result – a fully editable configuration surface. Step four is the trap: the bug is invisible locally and only surfaces when someone re-launches your task with different values, which is the worst possible time to find it.

An artifact is any file or Python object you attach to a task. Use them for the things that are neither metrics nor the final model: a preprocessed dataframe, an evaluation report, a vocabulary, a set of predictions.

PYTHON

Copy



```
# A local file or folder
task.upload_artifact("report", artifact_object="outputs/report.html")
task.upload_artifact("plots", artifact_object="outputs/plots/") # zipped

# Any picklable object
task.upload_artifact("vocab", artifact_object=vocab_dict)

# A numpy array
task.upload_artifact("embeddings", artifact_object=embeddings)
```

Reading them back from another script is the part that makes them useful:

consumer.py

Copy

```
from clearml import Task

producer = Task.get_task(project_name="vision", task_name="resnet18 baseline")
df = producer.artifacts["test predictions"].get() # deserialised object
path = producer.artifacts["report"].get_local_copy() # downloaded file path
```



PYTHON

Copy

```
task = Task.init(project_name="vision", task_name="run", output_uri="s3://my-bucket/clea
# or, for every task on this machine, in clearml.conf:
#   sdk.development.default_output_uri: "s3://my-bucket/clearml"
```

► TRY IT

1. Upload a small DataFrame as an artifact and open it in the UI's Artifacts tab, then note the inline preview.
2. Upload a folder and confirm it arrives as a single zip.
3. Write a second script that fetches the task by name and calls `.get()` on the dataframe artifact.
4. Check the artifact's URL in the UI and see which server it landed on.

Expected result – a preview you can read without downloading, and, in step three, data flowing between two runs with no shared filesystem. That is the mechanism pipelines are built on, so it is worth doing by hand once before you meet the abstraction.

Models and the registry: weights with provenance

A model is an artifact with privileges: its own registry entry, its own tags, and a recorded link back to the task that produced it. Most of the time you get one for free.



1 Created

A training task saves weights; a model appears, linked to that task.

2 Tagged

You add **candidate** , **staging** , or **production** . Tags are how humans and scripts find it.

3 Published

Marks it read-only. Weights and metadata can no longer be changed.

4 Consumed



3 Archived

Out of the default views, still fully queryable by id.

Query by tag, never by path

`InputModel(project="vision", name="resnet18-cls", tags=["production"])` is the whole point of a registry: your serving code stops containing a filename and starts containing an intent. Promoting a new model then means moving a tag rather than editing and redeploying code.

► TRY IT

1. Save a model with your framework's normal save call and find it in the project's Models list without writing any ClearML model code.
2. Open the model and follow the link back to the task that created it. Confirm you can reach the exact commit from there.
3. Tag it `candidate`, then write a script that loads it with `InputModel(..., tags=["candidate"])` and prints the local path.
4. Publish it, then try to change its tags or weights.

Expected result – a model whose provenance is one click from the weights, and a loader that names an intent rather than a path. Step four shows you what "published" enforces, which is worth knowing before you rely on it as a promotion gate.

Datasets: immutable, incremental versions

A tracked experiment that reads `/data/train.csv` is only half-reproducible. `ClearML Dataset` closes that gap: an immutable, versioned, content-addressed collection of files that is itself a task, so it has an id, a project, tags, and lineage.

From the command line:



```
clearml-data create --project datasets --name iris --version 1.0.0
clearml-data add --files ./data
clearml-data close # uploads and makes the version immutable
```

Or from Python, which is what you will use inside a pipeline:

PYTHON

Copy

```
from clearml import Dataset

ds = Dataset.create(dataset_project="datasets", dataset_name="iris", dataset_version="1.0.0")
ds.add_files("./data")
ds.upload()
ds.finalize()
print(ds.id)
```

Consuming it is one line, and this is the line that makes an experiment reproducible:

train.py

Copy

```
from clearml import Dataset

path = Dataset.get(dataset_project="datasets", dataset_name="iris", alias="training data")
df = pd.read_csv(f"{path}/train.csv")
```

`get_local_copy()` downloads once and caches. Call it again on the same machine and it returns the cached path immediately. The `alias` argument is important: it records the dataset id into the consuming task's parameters, so the experiment's Configuration tab names the exact data version it used.

Versions are **incremental** rather than full copies, which is what makes them cheap:

PYTHON

Copy

```
child = Dataset.create(
    dataset_project="datasets",
    dataset_name="iris",
```



```
child.add_files("./new-batch")    # only the delta is uploaded  
child.finalize()
```

Use `get_local_copy()` , not `get_mutable_local_copy()` , in training

The cached read-only copy is shared between every task on the machine, so ten runs against the same dataset download it once. A mutable copy is a fresh full extraction



► TRY IT

1. Create a dataset from a folder with two small CSVs and finalize it.
2. Create a child version that adds one more file, and confirm in the UI that only the new file was uploaded.
3. Fetch it in a training script with an `alias`, then check the training task's Configuration tab for the recorded dataset id.
4. Run the same fetch twice and compare the wall-clock time.

Expected result – a delta upload rather than a full copy, an instant second fetch from cache, and, the important one, a training task that records which data version it read. That recorded id is the difference between "trained on the customer data" and a reproducible claim.



✓ What you use it for

- **Details:** a diff of hyperparameters, packages, and even the code
- **Scalars:** all runs' curves overlaid on one chart
- **Plots:** side-by-side confusion matrices and custom figures

✗ What it will not do for you

- Compare metrics you only `print()` ed
- Compare parameters you never `connect` ed
- Diff data you read from an unversioned path

The same queries work from Python, which is how you build a report or a promotion gate:

PYTHON

Copy

```
from clearml import Task

tasks = Task.get_tasks(
    project_name="vision",
    task_filter={"status": ["completed"], "order_by": ["-last_update"]},
)

for t in tasks[:10]:
    metrics = t.get_last_scalar_metrics()
    print(t.name, metrics.get("accuracy", {}).get("val", {}).get("last"))

best = max(tasks, key=lambda t: t.get_last_scalar_metrics().get("accuracy", {}).get("val", {}).get("last", 0))
print("best:", best.id, best.name)
```

Tags are your only real organisation tool: use them from day one

Projects are folders; tags are cross-cutting. `baseline` , `ablation` , `broken` , `paper-v2` , `candidate` . These are what make a table of four hundred runs navigable a month later. Add them in code with `task.add_tags([...])` so they are never forgotten.



An **agent** is a process that watches a **queue**. When a task is enqueued, the agent claims it, rebuilds its environment from the recorded packages, clones the recorded git commit, applies the recorded diff, and runs the recorded entry point.

BASH

Copy

```
pip install clearml-agent
clearml-agent init                # same credentials flow as clearml-init

# Run a worker that watches the "default" queue
clearml-agent daemon --queue default

# On a GPU box: one worker per GPU
clearml-agent daemon --queue gpu --gpus 0
clearml-agent daemon --queue gpu --gpus 1

# In Docker mode - the environment is a container, not a virtualenv
clearml-agent daemon --queue gpu --docker nvidia/cuda:12.1.0-runtime-ubuntu22.04
```

CLONE

in the UI



EDIT

parameters



ENQUEUE

pick a queue



AGENT

rebuilds and runs



01 Right-click a completed task → Clone

You get an identical task in **draft** status. The original is untouched; a completed task is read-only.

02 Edit whatever you want

Hyperparameters, the docker image, the git branch, even the entry-point arguments. All editable while in draft.

03 Enqueue it, choosing a queue

The task moves to **queued**. If no agent watches that queue it waits, which is a normal state and not an error.

04 The agent picks it up

Creates a virtualenv (or container), installs the recorded packages, clones the commit, applies the diff, runs the script.

05 Watch it in the Console tab

Identical experience to a local run, on hardware you never logged into.

train.py

Copy

```
from clearml import Task

task = Task.init(project_name="vision", task_name="resnet18")
params = task.connect({"lr": 3e-4, "epochs": 50})

# Everything above this line runs locally: the task is created and configured.
# Then the local process exits and the task is enqueued for an agent.
task.execute_remotely(queue_name="gpu")

# Nothing below here runs locally – only on the agent.
train(**params)
```



line and the identical script runs locally for debugging. That is the pattern to learn.

Task status	Means
<code>draft</code>	Created or cloned, fully editable, not running
<code>queued</code>	Waiting for an agent on some queue
<code>in_progress</code>	An agent (or your laptop) is running it
<code>completed</code>	Finished with exit code 0
<code>failed</code>	Non-zero exit; the console log holds the traceback
<code>aborted</code>	Stopped by a user or by a <code>task.mark_stopped()</code>
<code>published</code>	Read-only and protected from cloning-over

► TRY IT

1. Start an agent on your own laptop: `clearml-agent daemon --queue default` . It counts as a worker.
2. Clone one of your completed tasks in the UI, change one hyperparameter, and enqueue it to `default` .
3. Watch the agent's terminal build a virtualenv, then watch the Console tab in the UI.
4. Add `task.execute_remotely(queue_name="default")` to your script and run it. Note how fast the local process exits.
5. Now enqueue a task whose commit you have not pushed, and read the failure.



concrete. You can watch both halves at once, and step five teaches you the one error you will otherwise spend an hour on.

Your first pipeline: tasks that create tasks

Once runs are re-executable, chaining them is a small step. A pipeline is a task that creates and monitors other tasks.

The decorator style is the clearest starting point:

pipeline.py

Copy

```
from clearml import PipelineDecorator

@PipelineDecorator.component(return_values=["data_path"], cache=True, execution_queue="cpu")
def prepare(dataset_name: str):
    from clearml import Dataset
    return Dataset.get(dataset_project="datasets", dataset_name=dataset_name).get_local_path()

@PipelineDecorator.component(return_values=["model_path", "accuracy"], execution_queue="cpu")
def train(data_path: str, lr: float, epochs: int):
    # ordinary training code
    return "model.pt", 0.941

@PipelineDecorator.component(return_values=["report"], execution_queue="cpu")
def evaluate(model_path: str, data_path: str):
    return {"auc": 0.97}

@PipelineDecorator.pipeline(name="train-and-eval", project="vision", version="1.0.0")
def main(dataset_name="iris", lr=3e-4, epochs=20):
    data_path = prepare(dataset_name)
    model_path, accuracy = train(data_path, lr, epochs)
    if accuracy > 0.9:
        report = evaluate(model_path, data_path)
        print(report)

if __name__ == "__main__":
```



A pipeline step here is literally "clone this existing task, override these parameters, run it". Nothing new is invented. It is the clone-and-enqueue loop from the previous section,



`run_locally()` first, always

Debugging a distributed pipeline is unpleasant. `PipelineDecorator.run_locally()` runs every component in your own process, sequentially, so a traceback is a normal traceback. Get it green locally, then delete that one line to fan it out across queues.

► TRY IT

1. Write a three-component pipeline where the first returns a number, the second doubles it, and the third prints it. Run it with `run_locally()`.
2. Remove `run_locally()` and run it against your local agent. Watch three separate tasks appear.
3. Open the pipeline task's Results → Pipeline tab and read the DAG it drew.
4. Rerun the pipeline unchanged and watch the `cache=True` component get skipped.

Expected result – a DAG you never drew and, in step four, a skipped step. Caching keys on the component's code and inputs, so an unchanged step is reused, which is what makes iterating on the last step of a long pipeline bearable.



Five failures cover most of what you will hit at this level:

Symptom	Cause	Fix
No metrics appear	<code>Task.init</code> ran after the framework import	Move it to the top of the entry point
Task stuck in <code>queued</code>	No agent on that queue	Start an agent, or check the queue name
Agent fails with a git error	Your commit was never pushed	Push the branch, then enqueue
<code>ImportError</code> on the agent only	Package imported lazily, so never detected	Add it to the task's package list or a <code>requirements.txt</code>
Parameter change had no effect	Code reads the literal, not <code>connect</code> 's return	Read from the returned object

useful in a debugging session

Copy



```
t = Task.get_task(task_id= '812c4b17e0a70311c0b0a2e4d709113b' )
print(t.status, t.get_last_iteration())
print(t.get_parameters())           # what it actually ran with
print(t.get_last_scalar_metrics())  # what it produced
print(t.data.script.diff[:500])     # the uncommitted patch it applied
t.mark_stopped()                   # abort a stuck run
```

► TRY IT: CAUSE EACH FAILURE ON PURPOSE

1. Move `Task.init` below your framework imports and confirm the scalars go missing.
2. Enqueue a task to a queue name that does not exist and watch it sit in `queued`.
3. Import a package inside a function only, then run the task on an agent and read the `ImportError`.
4. For each one, get to the diagnosis using only the UI tabs in the order above.

Expected result – three recognisable failures, two of which are silent rather than loud. The lazy-import one in step three is the sneakiest: it works on your machine and fails only on the agent, because only your machine already had the package.

Putting it all together



src/train.py

Copy

```
from clearml import Task                                # 1. import first

task = Task.init(                                       # 2. init before heavy imports
    project_name="vision",
    task_name="resnet18 baseline",
    output_uri="s3://my-bucket/clearml",               # 3. artifacts to object storage
)
task.add_tags(["baseline"])                             # 4. taggable from day one

import torch
from clearml import Dataset, OutputModel

params = task.connect({                                # 5. editable on a clone
    "lr": 3e-4,
    "batch_size": 64,
    "epochs": 20,
    "seed": 42,
})
cfg = task.connect_configuration("configs/model.yaml", name="model config")

# 6. remote execution is one line; comment it out to debug locally
task.execute_remotely(queue_name="gpu")
```



```
dataset_project="datasets",
dataset_name="iris",
alias="training data",
).get_local_copy()

logger = task.get_logger()
for epoch in range(params["epochs"]):
    train_loss = train_one_epoch(data, lr=params["lr"])
    val_loss, val_acc = validate(data)
    logger.report_scalar("loss", "train", train_loss, epoch)      # 8. curves
    logger.report_scalar("loss", "val", val_loss, epoch)
    logger.report_scalar("accuracy", "val", val_acc, epoch)

test_acc = test(data)
logger.report_single_value("test_accuracy", test_acc)           # 9. comparable
task.upload_artifact("predictions", predictions_df)             # 10. artifact

model = OutputModel(task=task, name="resnet18-cls", framework="PyTorch")
model.update_weights(weights_filename="model.pt")               # 11. registry
model.tags = ["candidate"]
```



```
# One-time setup
pip install clearml clearml-agent
clearml-init

# Everyday loop
python src/train.py                                # tracked; enqueued to gpu by execute_remotely
# → compare in the UI, clone the winner, edit lr, enqueue again

# A worker, wherever the hardware is
clearml-agent daemon --queue gpu --docker nvidia/cuda:12.1.0-runtime-ubuntu22.04
```

Eleven decisions in there are the whole lesson of this page:

Decision	Section
<code>Task.init</code> before framework imports	Your first tracked task
<code>output_uri</code> to S3, not the file server	Artifacts
Tags added in code, not by hand	Comparing experiments
Hyperparameters through <code>connect</code> , read from the return	Hyperparameters and configuration
A YAML config through <code>connect_configuration</code>	Hyperparameters and configuration
<code>execute_remotely</code> as the one line that moves the run	Agents and queues
Data through <code>Dataset.get(..., alias=...)</code>	Datasets
Curves with one <code>title</code> and several <code>series</code>	Logging on purpose
Final numbers as <code>report_single_value</code>	Logging on purpose
Extra outputs as artifacts, not loose files	Artifacts
Weights registered as a model, with a tag	Models and the registry



What you can now do, and what comes next

You can track an experiment with two lines, log metrics and artifacts deliberately, register and query models, version datasets incrementally, compare dozens of runs in a table and in code, clone a run and re-launch it on other hardware without SSH, chain runs into a cached pipeline, and diagnose a failed task in a fixed order. That is a working practitioner's toolkit, enough to own the experiment-tracking and reproducibility story on a real project.

Can you...	
Say what a Task is?	One recorded, re-executable run
Name the four parts of ClearML?	SDK, API/web/file server, agent, queue
Say why <code>Task.init</code> goes first?	It patches frameworks at import time
List what is captured automatically?	Code, diff, packages, args, TB metrics, models, console
Explain <code>title</code> versus <code>series</code> ?	One chart per title, one line per series
Say when to use <code>report_single_value</code> ?	Facts about the run, for comparison columns



Explain why <code>connect</code> is bidirectional:	Local upload, on an agent, download and override
Say where artifacts go by default?	The file server: set <code>output_uri</code> for anything large
Name the two dataset fetch modes?	<code>get_local_copy</code> cached, <code>get_mutable_local_copy</code> fresh
Say what an agent does with a queued task?	Rebuilds env, clones commit, applies diff, runs it
Explain what <code>execute_remotely</code> does?	Registers and enqueues, then exits locally
Name the first tab to open on a failure?	Console, then Execution

Mid-level takes every one of those topics further: hyperparameter optimisation with `HyperParameterOptimizer` and Optuna, task types and the services queue, agent modes and caching in depth, docker mode and `clearml-task` for launching unmodified code, remote debugging with `clearml-session`, dataset internals and squashing, the model registry as a promotion workflow, ClearML Serving, autoscalers, reports, and the CI patterns that make a training run a pull-request check.

Senior then covers what you own when ClearML is your team's platform: the self-hosted deployment and its three data stores, the access and credential model, multi-tenancy across teams, storage cost and retention, Elasticsearch and MongoDB scaling, upgrade and backup procedure, audit and lineage for regulated work, GPU fleet economics, incident playbooks, and where ClearML stops and a feature store, a data warehouse, or a dedicated serving stack begins.

NEXT UP

Beginner · Interview Review





MLOps MENA
COMMUNITY

العربية العربية



[Join the Community ↗](#)



MLOps MENA
COMMUNITY

Free MLOps and AI learning for engineers across the Middle East and North Africa — live sessions, three structured roadmaps, open cohort courses, study groups, and mentorship. We also train, staff, and build for companies across the region.



LEARN

[Roadmaps](#)

[Student Guides](#)

[Courses](#)

[The MLOps Practitioner](#)

[Sessions](#)

[Articles](#)

COMMUNITY

[Meet the team](#)

[Services for companies](#)

[FAQ](#)

[Contact us](#)

[Privacy Policy](#)

WHERE WE ARE

[WhatsApp](#)

[Discord](#)

[LinkedIn](#)

[X](#)

[YouTube](#)

[GitHub](#)



MLOps MENA
COMMUNITY

العربية



Join the Community ↗



Learning space

Free to join this week with code **FREECORETEAM**

© 2026 MLOps MENA Community. Free learning for the community.

Built by the community, for the community.